System for Artificial Intelligence

Assignment 1: CSA forward

Zijun Yin
Shanghai Innovation Institute

Why CSA?

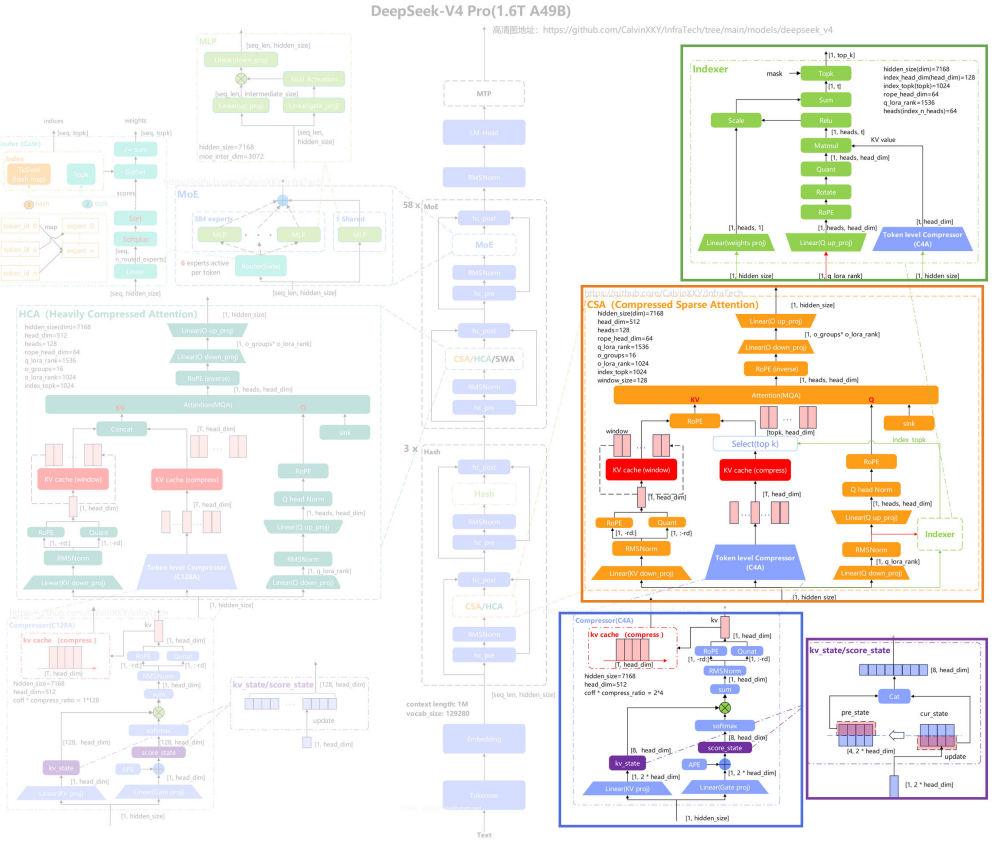
上海创智学院
Shanghai Innovation Institute

学 Infra，需要理解模型在算什么

模型的计算特性推动推理框架、训练框架等Infra组件的开发和演进

CSA 足够有挑战

涵盖 Indexer、压缩表示与稀疏注意力，学懂这些，再看其他模型就像遇见熟人



DeepSeek-V4 整体架构

https://github.com/CalvinXKY/InfraTech/tree/main/models/deepseek_v4

上海创智学院
Shanghai Innovation Institute

为了能 **深入体验** CSA 设计如下作业：

用纯 NumPy 从零实现 DeepSeek-V4 的压缩稀疏注意力



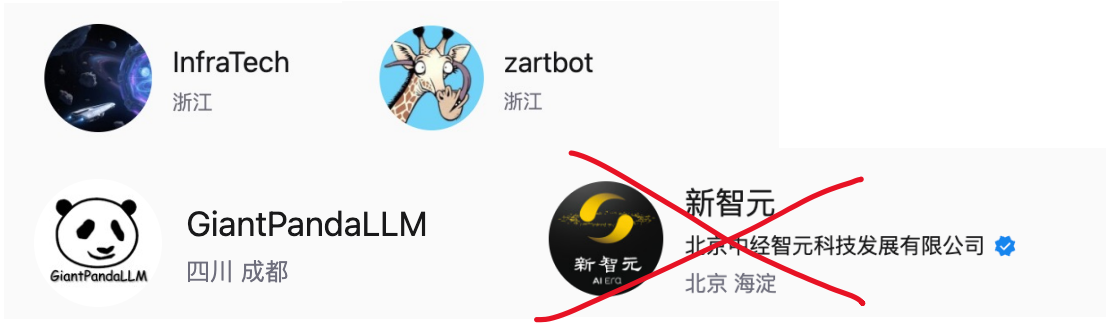
Compressed Sparse Attention

- 阶段一：手写 CSA 前向与反向
- 阶段二：在存储约束下完成融合计算

那这个时候有人就要问了：

我都没学过 CSA，怎么写...

1. 去各大公众号刷一下，一定能找到别人画好的架构图



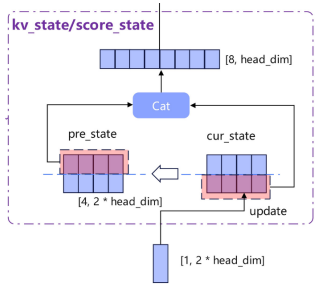
2. 想象自己是一个 token： $x \in R^{1xd}$ ，你从入口走进这个架构图的时候，在每一个模块里面会经历怎么样的变化 （源码对照公式）

下标 $p = 0, \ldots, 2m-1$ 是条目内的位置：前 m 个（$p < m$）来自前一块 $j-1$，后 m 个（$p \geq m$）来自本块 j。这两个数组只是把已有的数重新排列，图 5 里的 kv_state 和 score_state 就是它们。代码里没有现成的这两个变量，需要按下式用切片自己拼出来：

$$\texttt{values}_{b,j,p,c} = \begin{cases} \texttt{kv}_{b,(j-1)m+p,c}, & p < m,\ j \geq 1 \quad \text{（取块}j-1\text{ 的前半通道）}, \\ 0, & p < m,\ j = 0, \\ \texttt{kv}_{b,jm+p-m,D_c+c}, & p \geq m \quad \text{（取块}j\text{ 的后半通道）}, \end{cases}$$

$$\texttt{logits}_{b,j,p,c} = \begin{cases} \texttt{gate}_{b,(j-1)m+p,c}, & p < m,\ j \geq 1, \\ -\infty, & p < m,\ j = 0, \\ \texttt{gate}_{b,jm+p-m,D_c+c}, & p \geq m. \end{cases}$$

第 0 个条目没有前一块，所以前 m 个位置的门控 logit 设为 $-\infty$，softmax 后权重自然为 0，实际上只池化块 0 自己的 m 个位置。反过来看，除最后一个完整块的前半通道（本来要留给块 N，但块 N 还没凑齐）被丢弃外，前 $N \cdot m$ 个 token（即 N 个完整块里的全部 token）的其余通道都恰好落进唯一一个条目里（图 6）。



3. 现在有很多个 token，组成 $X \in R^{nxd}$，刚刚的行为有会有什么变化，哪些计算可以并行，哪些地方需要同步...

csa_lecture.pdf + reference/deepseek_v4/

4. （只有作业才需要考虑）（正常pytorch帮你做了）正向走通以后，反向怎么走？

recap：自动求导的本质是 算清每个变量的变化，会怎样影响最终损失

$$\frac{\partial L}{\partial x_i} = \sum_j \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial x_i}$$

其中 y = f(x)，L 为标量损失

解释： 把某个变量稍微改一点，最终的 loss 会增大还是减小，变化有多明显

做法：从输出梯度出发，**逐层应用链式法则**，并累加各分支的梯度。

矩阵求导补充

省流版（无偏置线性层）

$$Y = XW^{\mathrm{T}} \quad G = \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial X} = GW \quad \frac{\partial L}{\partial W} = G^{\mathrm{T}}X$$

backward(dy)：传进来的是什么?

$$\mathrm{dy} = \frac{\partial L}{\partial y}$$

损失对本模块输出的梯度，由后续计算传回；与 y 同形状。

CSA Query 分支的调用顺序

```
# 前向：保存反向要用的数据
z = wq_a.forward(x)
q = q_norm.forward(z)
# 反向：dq 由后续模块传来
dz = q_norm.backward(dq)
dx = wq_a.backward(dz)
```

Linear 内部（二维输入）

```
def backward(self, dy):
    x, W = self._x, self.weight
    dW = dy.T @ x
    self._grads["weight"] += dW
    dx = dy @ W
    return dx
```

调用 wq_a.backward(dz) 时，实参 dz 传给形参 dy

读取：前向缓存 self._x 和层参数 self.weight

返回 dx 给前一层；把 dW 累加到参数梯度中，参数更新由 optimizer 完成

到这里，你就发现会写第一部分了

模块	前向	反向
Linear / RMS / RMSNorm	3	7
Compressor	8	15
Indexer	8	5
Sparse Attention	8	15
Attention	10	16
整层检查	共 5 分	

实现文件：csa/attention.py

评分机制：

数学正确性 100 分

Rules：

- 按模块分别检查
- 参数梯度累加进 _grads
- 遵守规定的模块调用
- 不原地修改输入
- 规则项最多扣 5 分

把公式推出来之后写阶段一，其实就是

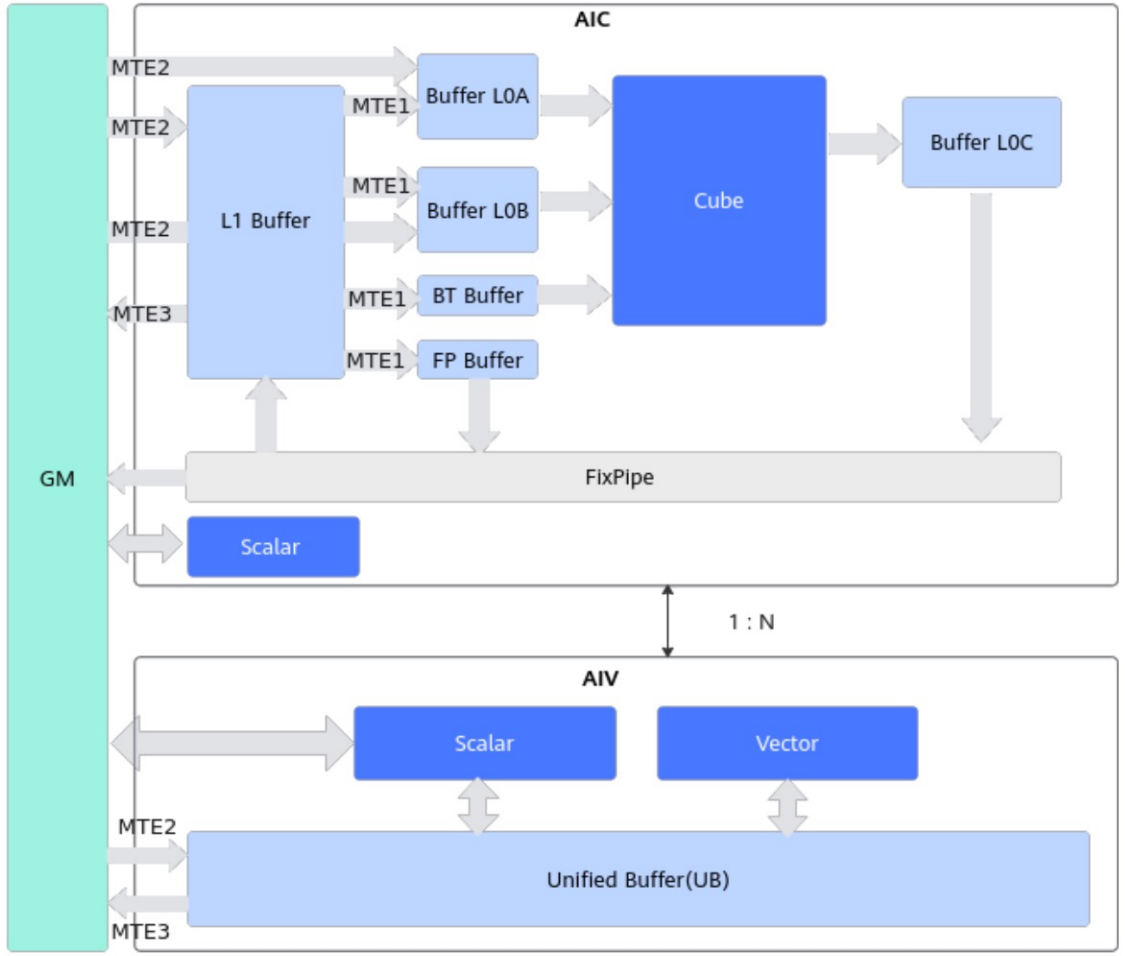
用 Numpy 翻译公式

不够好玩、

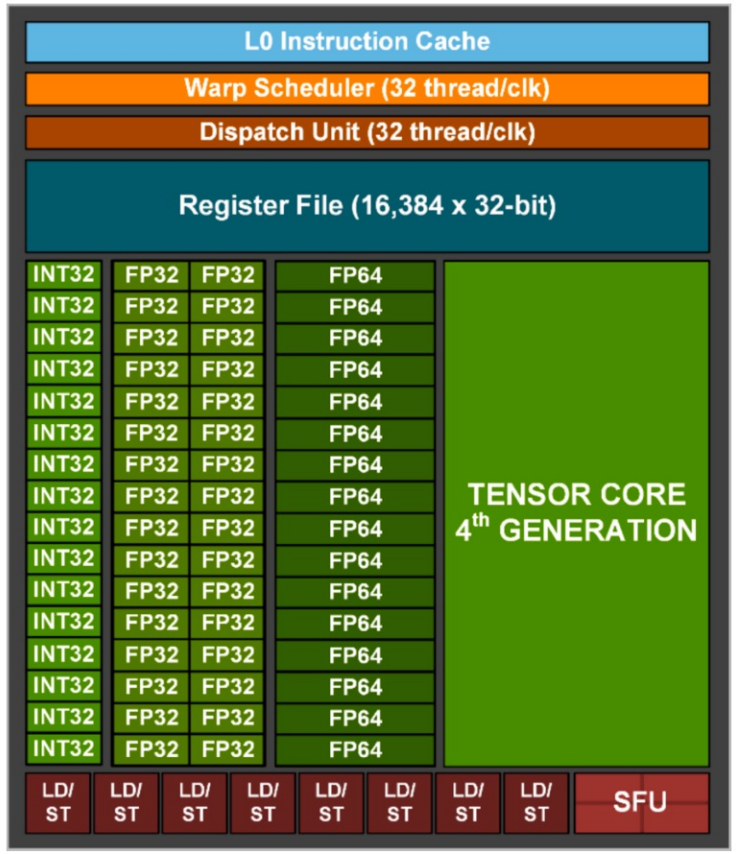


在真实硬件上还需要考虑这些...

NPU / Ascend AI Core



GPU



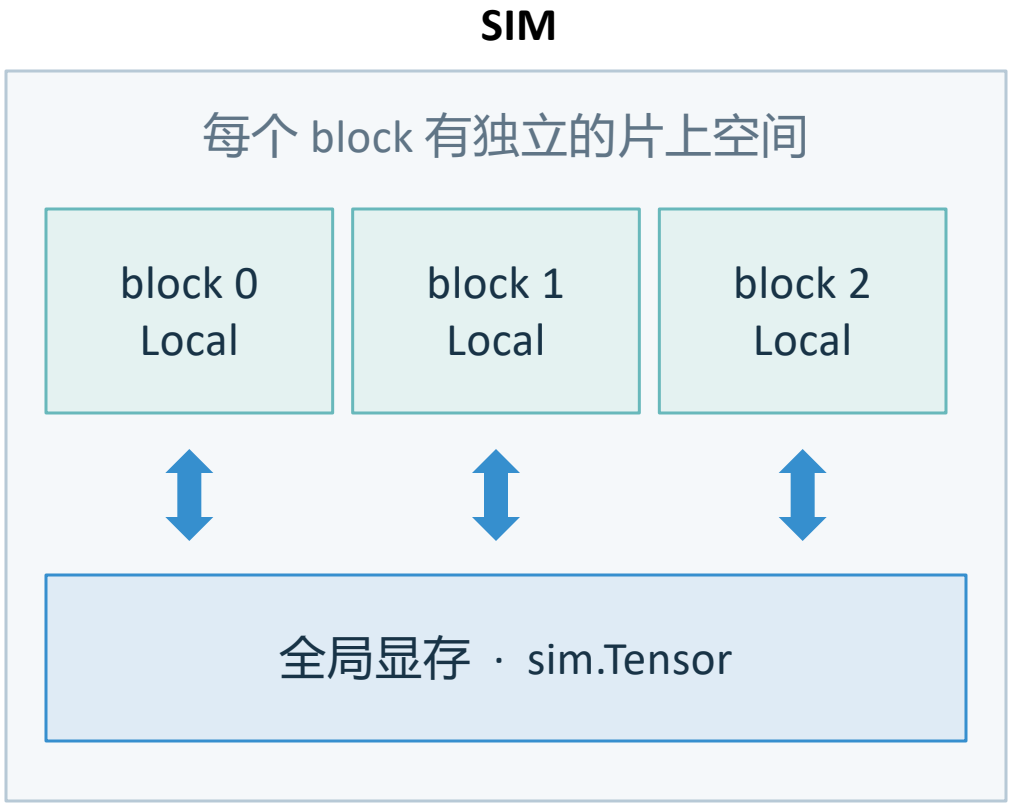
但由于各种原因没有加速卡...于是整了个**模拟器**

两级存储模型

sim.Tensor：全局显存
sim.Local：kernel 内的片上数组

sim.capacity()：每 block 48 KiB

同一时刻存活的 Local 都计入容量，
包括表达式产生的临时数组。



kernel 内：和片上存储交互

读到片上

sim.load / sim.gather

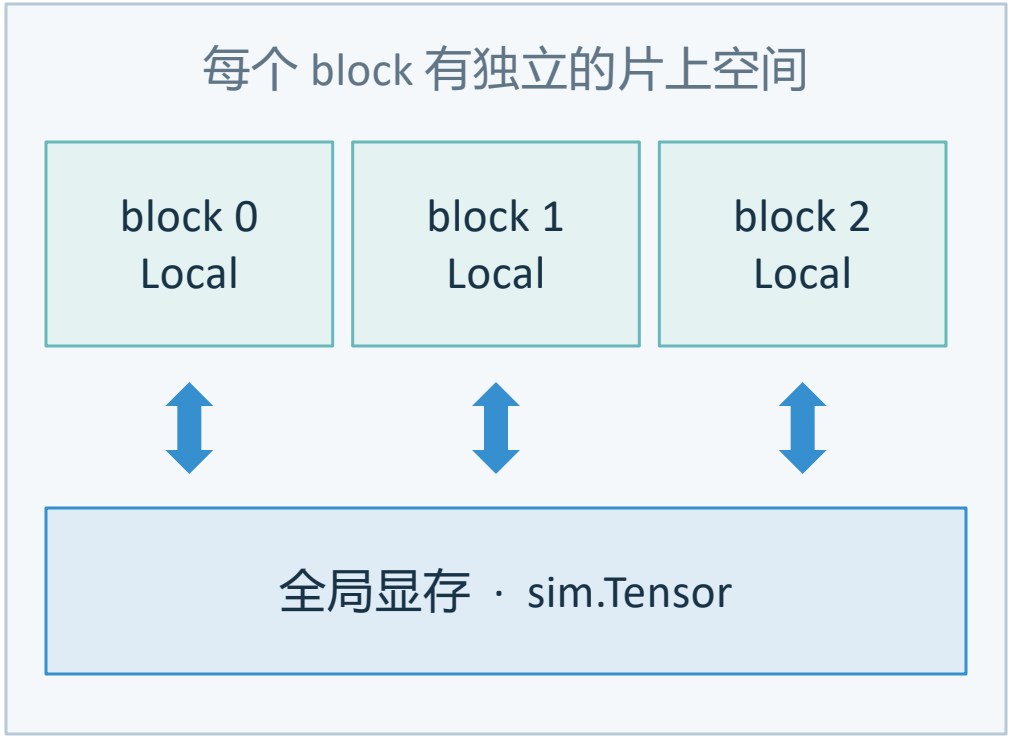
读取连续切片，或按索引读取行。

写回与累加

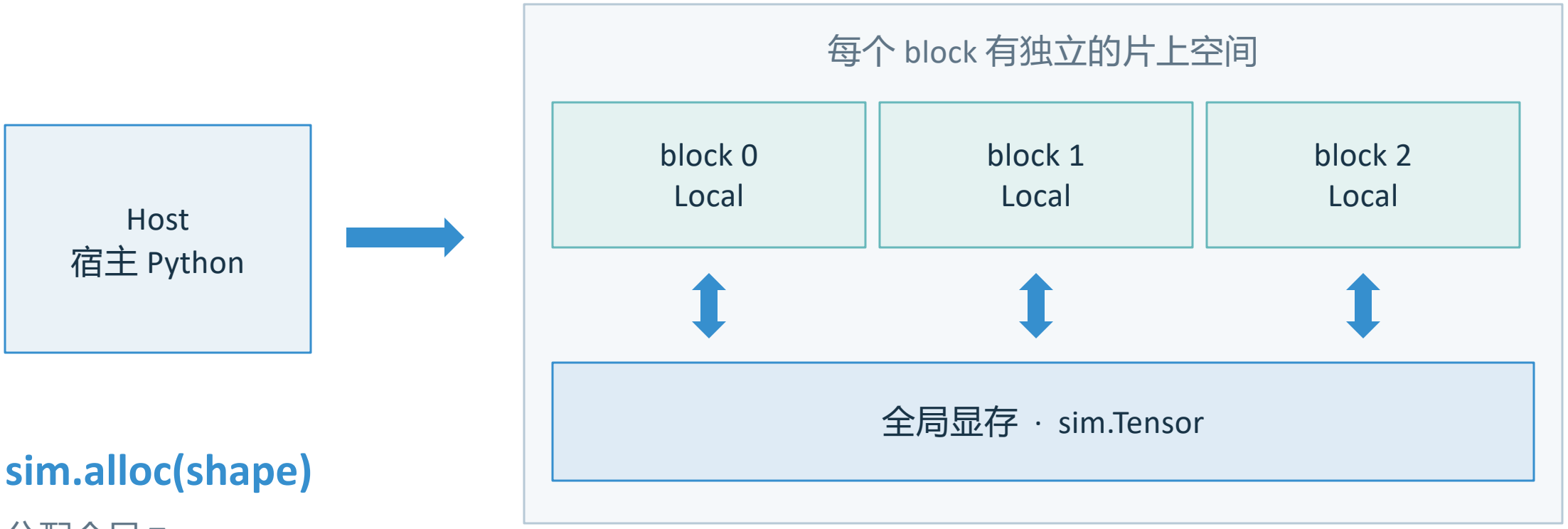
sim.store / sim.atomic_add
sim.scatter_add

中间计算用 NumPy 风格的数组运算。

创建 Local： sim.zeros / sim.ones / sim.full / sim.arange



宿主端：分配显存、启动计算



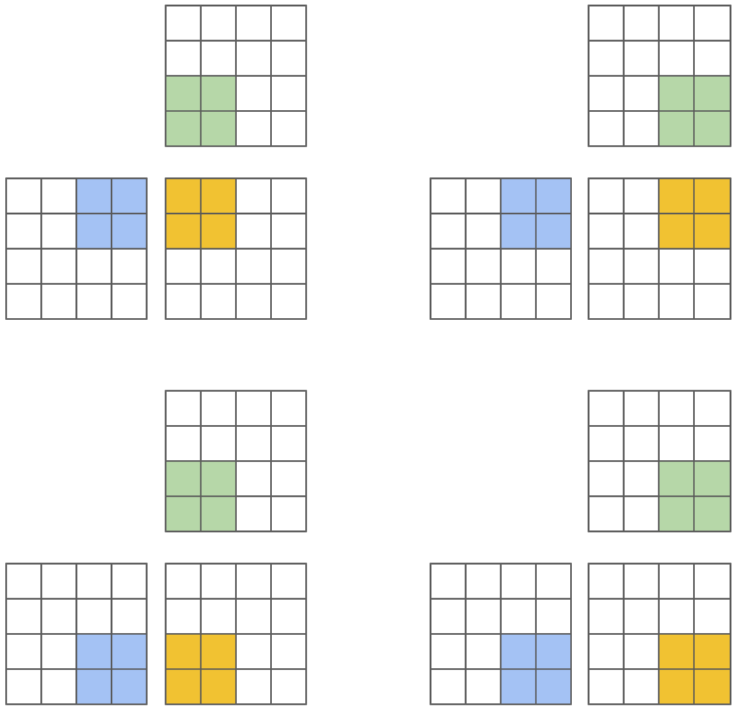
sim.alloc(shape)

分配全局 Tensor

sim.launch(kernel, grid, ...)

Tile 编程：一个实例处理一块数据

同一份 kernel，通过 pid 处理不同的数据块



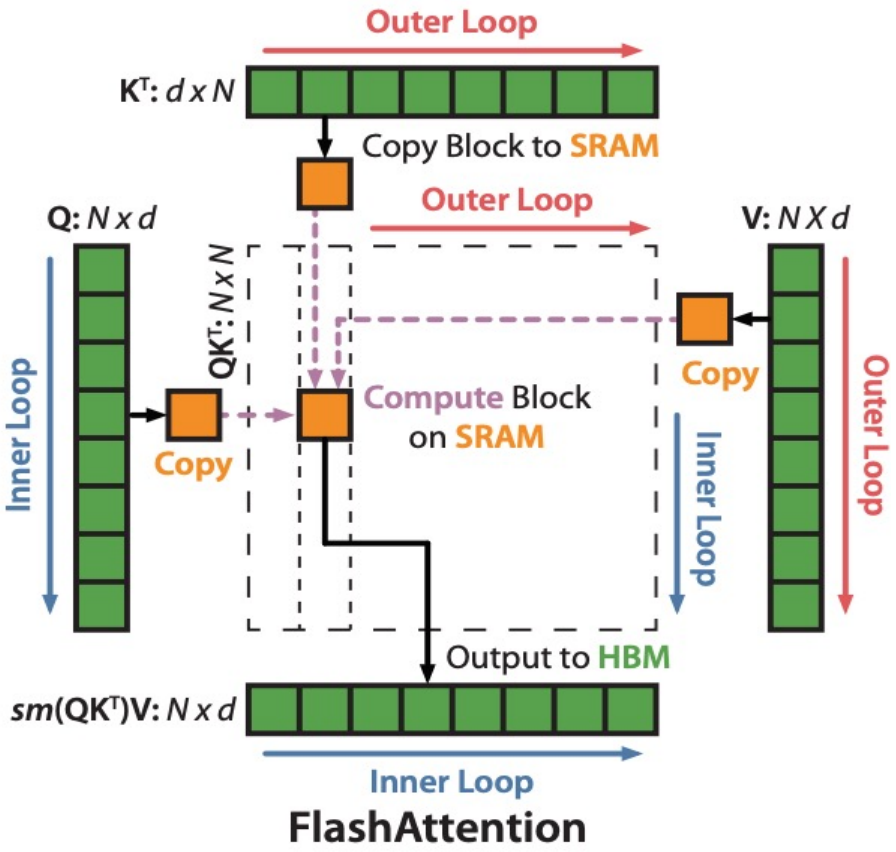
矩阵乘法示意：每个实例计算一个输出块

pid 确定范围，load 读入一个 tile。
在 Local 上批量计算，再 store 写回。

```
def kernel(pid, x, y, tile):
    start = pid * tile
    end = min(start + tile, x.shape[0])
    a = sim.load(x, slice(start, end))
    result = a * 2
    sim.store(y, slice(start, end), result)
```

https://triton-lang.org/main/_images/triton-parallel-matmul.png　推荐阅读：https://triton-lang.org/main/getting-started/tutorials/index.html

方案一：分块（Tiling）



Tips： sparse_attn 的分块计算

分批读入选中的 KV → 逐块计算注意力，

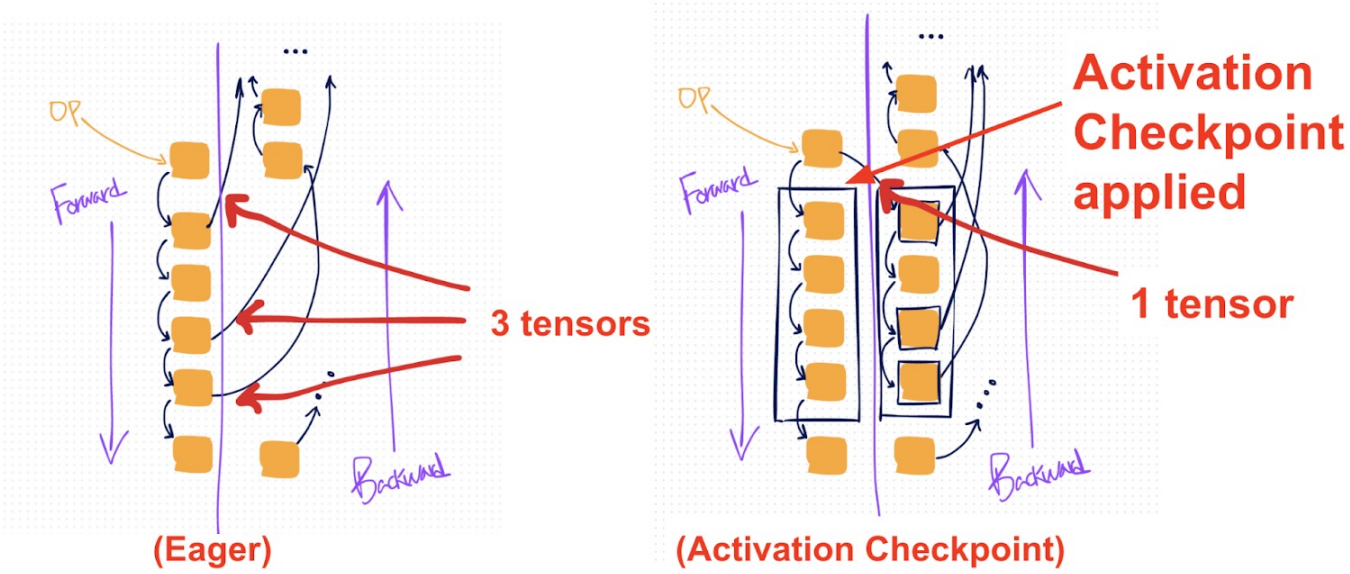
用在线 softmax 累积结果 → 最后写出 out

分块解决“装不下”；在线 softmax 保证分块后仍算对。

https://crfm.stanford.edu/2023/01/13/flashattention.html

方案二：需要时重算

前向少保存，反向再生成用得上的中间量



Tips：

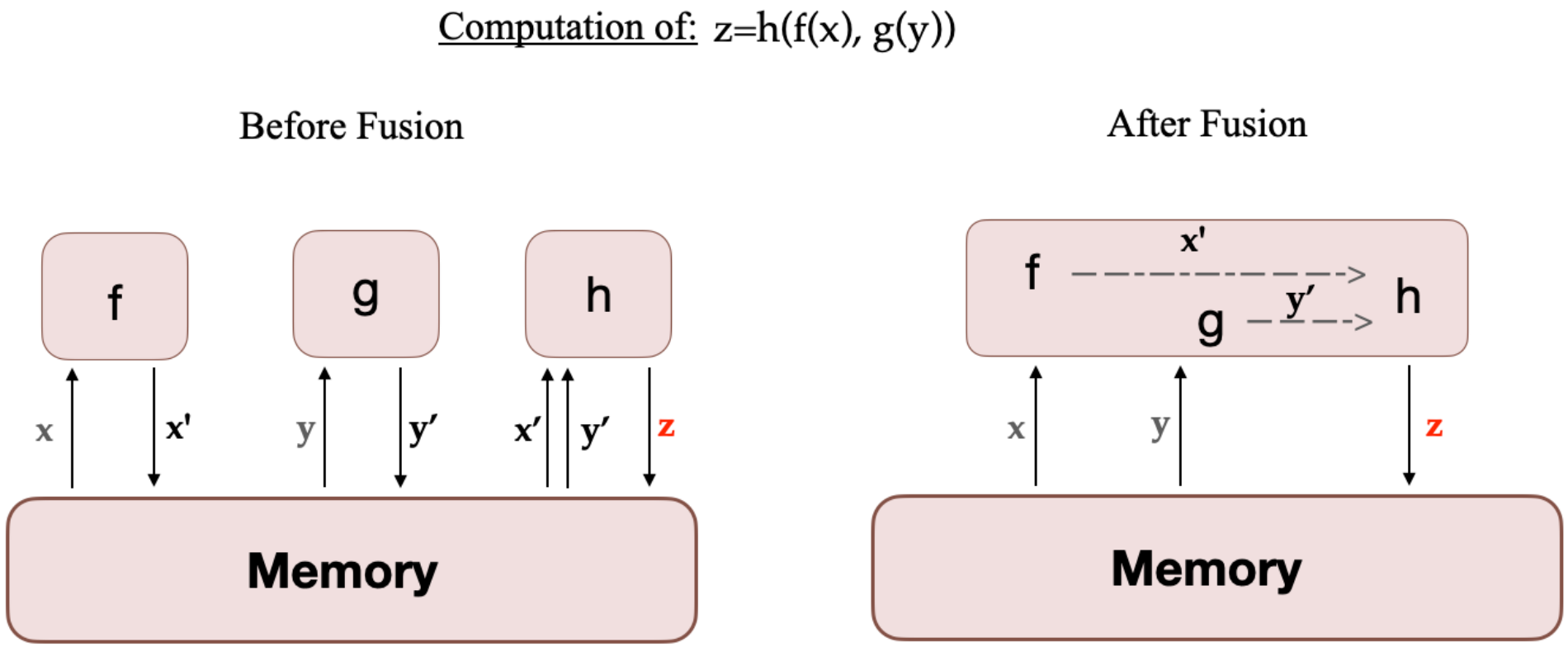
sparse_attn_backward 的概率重算

logit = softmax_scale * dot(q, kv)
p = exp(logit - lse)

前向保存每行的归一化统计量 lse → 反向按块重新计算注意力分数，

恢复概率 p → 用于计算梯度，用完即可释放。

方案三：中间结果留在片上



Tips： Indexer 的打分与 top-k 融合

分块算分数 → 立即 merge_topk → 只写出最终索引

图：Loubna Ben Allal / Hugging Face，How to train a Language Model with Megatron-LM（2022）

上海创智学院
Shanghai Innovation Institute

到这里，第二阶段你也会写了

实现函数	正确性	内存
sparse_attn	12	8
sparse_attn_backward	15	10
gated_pool	5	3
gated_pool_backward	7	5
index_topk	10	5
attention_forward	12	8

实现文件：csa/fused.py

评分机制：

正确性 61 分，内存 39 分

公开 40%，隐藏 60%

Rules：

- 结果正确后才检查内存
- 片上峰值不超过 48 KiB
- 流量、额外保存量均受限
- 整层仅要求前向

提交方式与截止时间

上海创智学院
Shanghai Innovation Institute

提交三个文件

attention.py、 fused.py、 feedback.txt

在 handout 目录运行：

uv run python -m zipfile -c 学号.zip csa/attention.py csa/fused.py feedback.txt

点击进入作业提交表

截止时间

2026 年 11 月 15 日 23:59:59